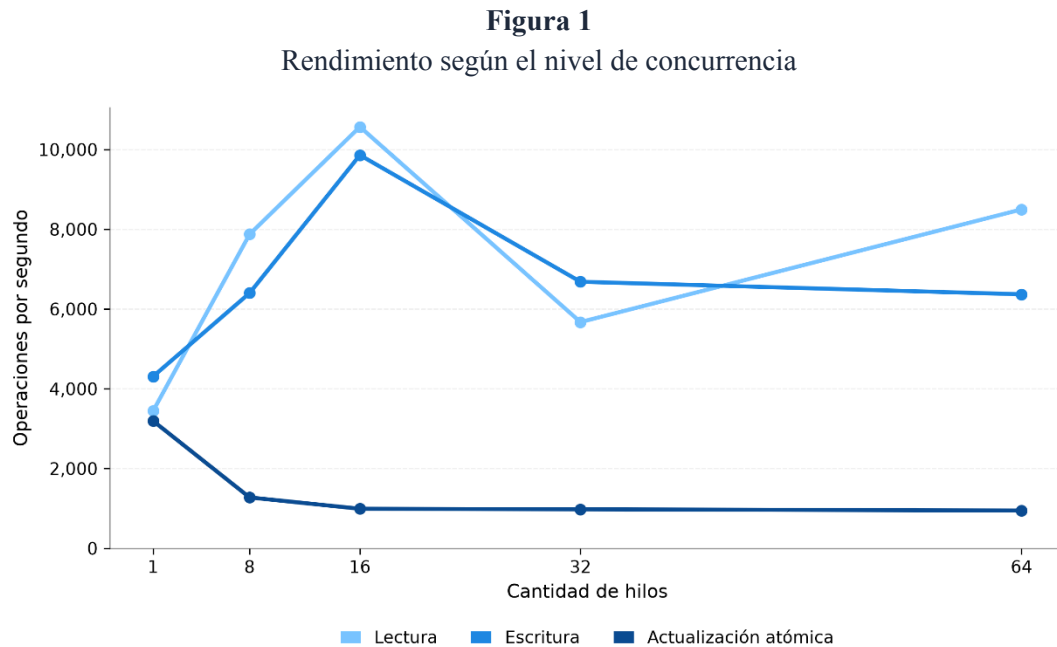


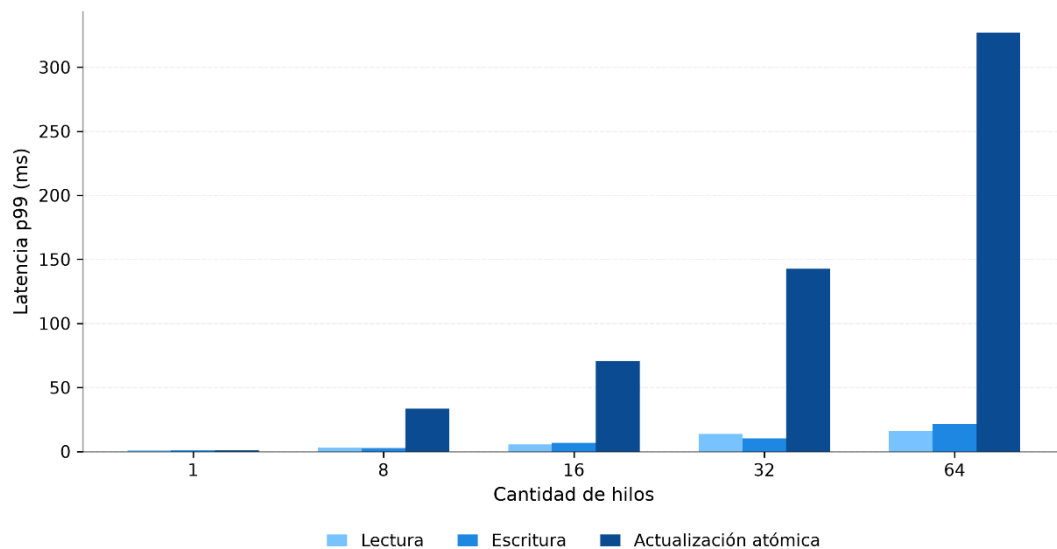
RESULTADOS

Para evaluar el comportamiento de la solución se realizaron pruebas de lectura, escritura y actualización atómica con 1, 8, 16, 32 y 64 hilos concurrentes. En cada escenario se ejecutaron 5000 operaciones y se midieron rendimiento y latencia. Además, se realizó una prueba general con 50000 usuarios simulados y 100000 intentos de reserva.



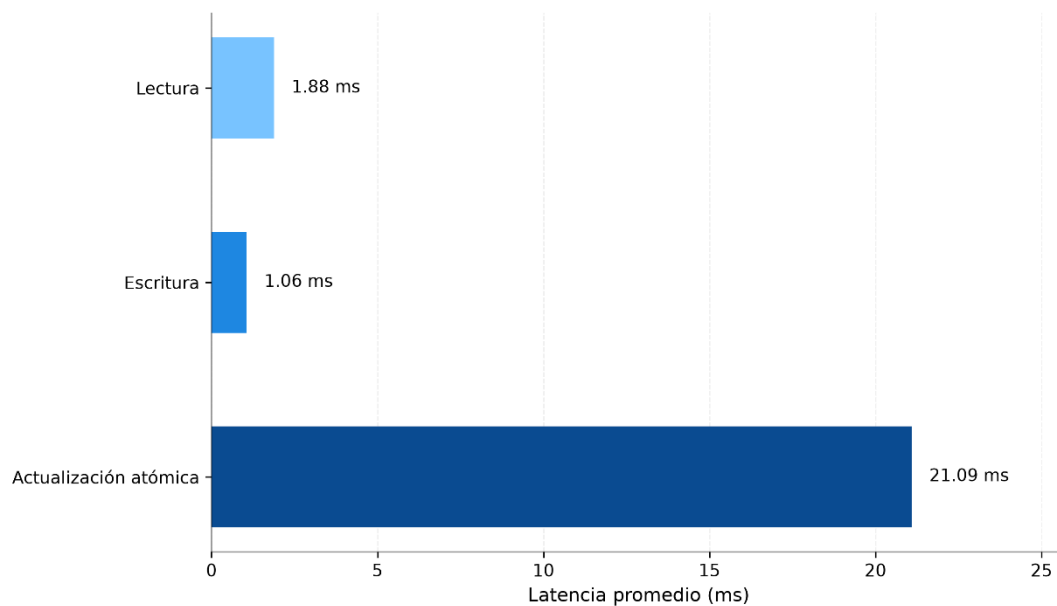
La Figura 1 muestra que las operaciones de lectura y escritura mejoran inicialmente al aumentar la concurrencia, alcanzando sus mejores resultados en niveles intermedios de hilos. A partir de ese punto, agregar más concurrencia deja de producir mejoras proporcionales. La actualización atómica presenta un comportamiento distinto, ya que su rendimiento disminuye conforme aumenta la cantidad de hilos debido a la competencia por modificar el mismo registro.

Figura 2
Latencia p99 según el nivel de concurrencia



La Figura 2 muestra el comportamiento de las operaciones más lentas. Mientras lectura y escritura mantienen un crecimiento moderado de la latencia, la actualización atómica alcanza aproximadamente 349 ms de p99 con 64 hilos. Aun así, en todos los escenarios se completaron las 5000 actualizaciones esperadas, por lo que el aumento de latencia no implicó pérdida de operaciones.

Figura 3
Latencia promedio de las operaciones con 32 hilos



Con 32 hilos, la escritura presentó una latencia promedio de 1.18 ms y la lectura de 1.99 ms, mientras que la actualización atómica alcanzó 22.16 ms. Esta diferencia evidencia el costo adicional de resolver conflictos cuando varias operaciones compiten por modificar un mismo dato.

La prueba general de carga se resume en la Tabla 1.

Tabla 1
Prueba general de carga

Métrica	Resultado
Usuarios simulados	50000
Intentos de reserva	100000
Hilos concurrentes	32
Solicitudes procesadas	5891.49
Reservas realizadas	800
Inventario final	0
Sobreventa	No

La prueba utilizó un inventario de 800 entradas frente a 100000 intentos de reserva. Una vez agotada la disponibilidad, las solicitudes restantes fueron rechazadas y el inventario terminó exactamente en cero, sin valores negativos ni sobreventa.

También se comprobó el funcionamiento de las reservas temporales mediante TTL, la devolución de inventario después de una expiración, la confirmación y cancelación de reservas, la recuperación de sesiones y el mecanismo de rate limiting.

HALLAZGOS

Los resultados muestran que aumentar la concurrencia puede mejorar inicialmente el rendimiento, pero este efecto no se mantiene de forma indefinida. En las operaciones de lectura y escritura, el rendimiento mejora al aumentar la concurrencia hasta alcanzar un nivel intermedio. A partir de ahí, el beneficio de agregar más hilos comienza a disminuir. Esto coincide con lo planteado por Kleppmann [1], quien señala que el desempeño debe analizarse según la carga de trabajo y considerando conjuntamente métricas como rendimiento y latencia.

El comportamiento de la actualización atómica responde a la competencia por un mismo registro. Cuando otro hilo modifica el dato antes de completar una operación, esta debe intentarse nuevamente. Este comportamiento es consistente con los principios del control optimista de concurrencia descritos por Kung y Robinson [3]. Aunque esto aumenta la latencia, las 5000 actualizaciones se completaron correctamente, por lo que el mecanismo mantuvo la consistencia aun bajo alta contención.

Los resultados también son coherentes con la orientación de Aerospike hacia cargas operacionales de baja latencia descrita por Srinivasan et al. [4]. Asimismo, la metodología sigue una idea similar a YCSB [5], al utilizar cargas controladas y métricas cuantitativas para comparar el comportamiento de las operaciones. En relación con las metas del proyecto, la solución procesó los 100000 intentos definidos, evitó la sobreventa y mantuvo las funcionalidades necesarias para gestionar reservas concurrentes.

LIMITACIONES

La principal limitación corresponde al entorno de ejecución. Las pruebas se realizaron localmente mediante Docker, por lo que los valores obtenidos dependen de los recursos disponibles en el equipo y no representan el rendimiento máximo de Aerospike en infraestructura dedicada. Además, los resultados corresponden a ejecuciones puntuales, por lo que pueden existir pequeñas variaciones entre una ejecución y otra.

También debe considerarse que la prueba de actualización atómica concentró todas las modificaciones sobre un mismo registro. Esto permitió generar una condición clara de contención, pero representa un escenario más exigente que una situación real, donde las solicitudes estarían distribuidas entre diferentes eventos y zonas.

Finalmente, las pruebas se realizaron sobre una instancia local y no sobre un clúster de múltiples nodos. Por esta razón, no se evaluaron directamente aspectos como replicación, distribución de carga, pérdida de conectividad o recuperación ante la caída de una instancia.

TRABAJO FUTURO

Como mejora, sería conveniente repetir cada escenario varias veces y utilizar medidas agregadas como promedio o mediana, con el fin de reducir el efecto de variaciones propias del entorno.

También sería útil ejecutar la solución sobre un clúster de Aerospike con varios nodos y utilizar cargas más cercanas a una preventiva real, distribuyendo las solicitudes entre distintos eventos y zonas. Esto permitiría evaluar escalamiento, replicación, recuperación ante fallos y diferentes niveles de contención bajo condiciones más próximas a un entorno de producción.

REFERENCIAS

- [1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [3] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, Jun. 1981.
- [4] V. Srinivasan et al., "Aerospike: Architecture of a Real-Time Operational DBMS," *Proc. VLDB Endow.*, vol. 9, no. 13, pp. 1389–1400, Sep. 2016.
- [5] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB", *Proceedings of the 1st ACM Symposium on Cloud Computing*, pp. 143–154, 2010.